Department of Computer Science and Engineering

University of Dhaka

Implementation of Interpolation Techniques: Lagrange Interpolation and Newton's Divided Difference Method

---

CSE 3212: Numerical Analysis Lab

Batch: 28 / 3rd Year 2nd Semester 2024

Submitted to:
Dr. Muhammad Ibrahim
Palash Roy

Submitted by:
Farzana Tasnim (14)

Contents

1 Introduction

1.1 The Lagrange Polynomial

Lagrange interpolation constructs an interpolating polynomial of degree n for given data points $(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$:

$$P_n(x) = \sum_{i=0}^{n} y_i L_i(x)$$

where

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}$$

1.1.1 Characteristics

- Simple to understand.
- No need to construct a difference table.
- Requires recomputation if a new data point is added.

1.2 Newton's Divided Difference Interpolation

Newton's interpolation uses a divided difference table to form an interpolation polynomial.

Divided Difference Table Formula

$$f[x_i, x_{i+1}] = \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}$$

$$f[x_i, x_{i+1}, \ldots, x_{i+k}] = \frac{f[x_{i+1}, \ldots, x_{i+k}] - f[x_i, \ldots, x_{i+k-1}]}{x_{i+k} - x_i}$$

Newton Polynomial

$$P_n(x) = f[x_0] + (x - x_0) f[x_0, x_1] + (x - x_0)(x - x_1) f[x_0, x_1, x_2] + \cdots$$

1.2.1 Characteristics

- Efficient for incremental addition of data points.
- Uses a difference table — easier for computation.
- Suitable for numerical algorithms.

1.3 Difference Between Lagrange and Newton Interpolation

Feature	**Lagrange Interpolation**	**Newton's Divided Difference**
Complexity	High for new data addition	Efficient for adding new points
Approach	Direct formula	Table-based recursive computation
Programming Ease	Simple	Slightly complex but scalable
Best Use	Small data sets	Large data sets & incremental updates

1.4 Comparison Table

Method	**Type**	**Derivative**	**Convergence**	**Speed**
Bisection	Bracketing	No	Linear	Slow
False Position	Bracketing	No	Linear	Moderate
Newton–Raphson	Open	Yes	Quadratic	Very Fast
Secant	Open	No	Superlinear	Moderate

Table 1: Comparison of Root Finding Methods

2 Objectives

- To understand the concept and need of **interpolation** for the estimation of unknown values.
- To implement **Lagrange Interpolation** to approximate values of a function using tabulated data.
- To implement **Newton's Divided Difference Interpolation** and construct the forward interpolation polynomial.
- To compare the **efficiency**, **computational complexity**, and **suitability** of both interpolation techniques.

3 Algorithms

3.1 Lagrange Interpolation

1. Read n and data points $(x[i], y[i])$.
2. Read the value of x for which $f(x)$ is required.
3. Initialize `result` $= 0$.
4. For $i = 0$ to $n - 1$:
   (a) Compute $L_i = 1$.
   (b) For $j = 0$ to $n - 1$, where $j \neq i$:

$$L_i = L_i \times \frac{(x - x[j])}{(x[i] - x[j])}$$

   (c) Update `result` $=$ `result` $+ L_i \times y[i]$.
5. Display `result`.

3.2 Newton's Divided Difference Interpolation

1. Read n and input data points $(x[i], y[i])$.
2. Construct the **divided difference table**.
3. Initialize `result` $= f[x_0]$.
4. For $k = 1$ to $n - 1$:
   (a) Set `term` $= f[x_0, x_1, \ldots, x_k]$.
   (b) For $j = 0$ to $k - 1$:

$$\texttt{term} = \texttt{term} \times (x - x[j])$$

   (c) Update `result` $=$ `result` $+$ `term`.
5. Display `result`.

4 Implementation

```python
def choose_nearest_nodes(x_nodes, y_nodes, x0, degree):
    k = degree + 1
    idx_sorted = np.argsort(np.abs(x_nodes - x0))
    chosen_idx = np.sort(idx_sorted[:k])
    return x_nodes[chosen_idx], y_nodes[chosen_idx]
```

Figure 1: Sorting Function

```python
def lagrange_eval(xi, yi, x):
    n = len(xi)
    total = 0.0
    for j in range(n):
        term = yi[j]
        for m in range(n):
            if m != j:
                term *= (x - xi[m]) / (xi[j] - xi[m])
        total += term
    return total
```

Figure 2: Lagrange implementation

```python
def newton_coeffs(xi, yi):
    n = len(xi)
    dd = np.zeros((n, n))
    dd[:, 0] = yi
    for j in range(1, n):
        for i in range(n - j):
            dd[i, j] = (dd[i + 1, j - 1] - dd[i, j - 1]) / (xi[i + j] - xi[i])
    return dd[0, :], xi
```

Figure 3: Newton coefficient implementation

```python
def newton_eval(coeffs, xi, x):
    n = len(coeffs)
    result = coeffs[-1]
    for k in range(n - 2, -1, -1):
        result = result * (x - xi[k]) + coeffs[k]
    return result
```

Figure 4: Newton Function

```python
def lagrange_poly_sym(xi, yi):
    x = symbols('x')
    n = len(xi)
    poly = 0
    for j in range(n):
        term = yi[j]
        for m in range(n):
            if m != j:
                term *= (x - xi[m]) / (xi[j] - xi[m])
        poly += term
    return simplify(expand(poly))
```

Figure 5: Lagrange polynomial

```python
def newton_poly_sym(coeffs, xi):
    x = symbols('x')
    n = len(coeffs)
    poly = coeffs[-1]
    for k in range(n - 2, -1, -1):
        poly = poly * (x - xi[k]) + coeffs[k]
    return simplify(expand(poly))
```

Figure 6: Newton polynomial

5 Output

```
Predicted temperatures at x = 45 km with absolute relative error Δ and points used:

Degree 2:
  Points used: [(35.0, 33.2), (50.0, 35.5), (65.0, 36.1)]
  Lagrange T(45) = 34.9222, Δ = 0.0000e+00
  Newton T(45)   = 34.9222, Δ = 0.0000e+00
  Lagrange Poly: -0.003777777777777777*x**2 + 0.474444444444444*x + 21.2222222222223
  Newton Poly:   -0.003777777777777777*x**2 + 0.474444444444444*x + 21.2222222222222

Degree 3:
  Points used: [(20.0, 29.4), (35.0, 33.2), (50.0, 35.5), (65.0, 36.1)]
  Lagrange T(45) = 34.9123, Δ = 9.8765e-03
  Newton T(45)   = 34.9123, Δ = 9.8765e-03
  Lagrange Poly: -9.87654320987598e-6*x**3 - 0.00229629629629638*x**2 + 0.402592592592594*x + 22.3456790123456
  Newton Poly:   -9.87654320987599e-6*x**3 - 0.00229629629629637*x**2 + 0.402592592592596*x + 22.3456790123456

Degree 4:
  Points used: [(20.0, 29.4), (35.0, 33.2), (50.0, 35.5), (65.0, 36.1), (80.0, 37.8)]
  Lagrange T(45) = 34.9741, Δ = 6.1728e-02
  Newton T(45)   = 34.9741, Δ = 6.1728e-02
  Lagrange Poly: 2.46913580246912e-6*x**4 - 0.000429629629629624*x**3 + 0.0230740740740735*x**2 - 0.237530864197517*x + 27.9629629629627
  Newton Poly:   2.46913580246912e-6*x**4 - 0.000429629629629626*x**3 + 0.0230740740740738*x**2 - 0.237530864197523*x + 27.9629629629629
```

Figure 7: Degree 2, 3, 4 Output

```
Degree 5:
  Points used: [(10.0, 26.7), (20.0, 29.4), (35.0, 33.2), (50.0, 35.5), (65.0, 36.1), (80.0, 37.8)]
  Lagrange T(45) = 34.9561, Δ = 1.7957e-02
  Newton T(45)   = 34.9561, Δ = 1.7957e-02
  Lagrange Poly: 2.05226871893537e-8*x**5 - 2.6615359948693e-6*x**4 + 6.03495270161758e-5*x**3 + 0.000883918550586116*x**2 + 0.234798781465433*x + 24.2278338945006
  Newton Poly:   2.05226871893533e-8*x**5 - 2.6615359948692e-6*x**4 + 6.03495270161829e-5*x**3 + 0.000883918550585611*x**2 + 0.234798781465442*x + 24.2278338945006

Degree 6:
  Points used: [(0.0, 25.0), (10.0, 26.7), (20.0, 29.4), (35.0, 33.2), (50.0, 35.5), (65.0, 36.1), (80.0, 37.8)]
  Lagrange T(45) = 34.9431, Δ = 1.2993e-02
  Newton T(45)   = 34.9431, Δ = 1.2993e-02
  Lagrange Poly: 4.24267090933731e-10*x**6 - 8.97867564534205e-8*x**5 + 8.52850852850863e-6*x**4 - 0.000499683033016363*x**3 + 0.0152358135691475*x**2 + 0.0599370999370976*
x + 25.0
  Newton Poly:   4.2426709093374e-10*x**6 - 8.97867564534193e-8*x**5 + 8.52850852850821e-6*x**4 - 0.000499683033016355*x**3 + 0.0152358135691467*x**2 + 0.0599370999371008*x
 + 25.0
```

Figure 8: Degree 5, 6 Output

```
  Degree 7:
    Points used: [(0.0, 25.0), (10.0, 26.7), (20.0, 29.4), (35.0, 33.2), (50.0, 35.5), (65.0, 36.1), (80.0, 37.8), (90.0, 38.9)]
    Lagrange T(45) = 34.9704, Δ = 2.7255e-02
    Newton T(45)   = 34.9704, Δ = 2.7255e-02
    Lagrange Poly: -1.977651977651944e-11*x**7 + 5.56616223282873e-9*x**6 - 6.11392465559102e-7*x**5 + 3.46335146335142e-5*x**4 - 0.00116867325575658*x**3 + 0.0233867061950388
  *x**2 + 0.0239438339438363*x + 25.0
    Newton Poly:   -1.977651977651944e-11*x**7 + 5.56616223282879e-9*x**6 - 6.11392465559119e-7*x**5 + 3.46335146335138e-5*x**4 - 0.00116867325575657*x**3 + 0.0233867061950392
  *x**2 + 0.0239438339438355*x + 25.0

  Degree 8:
    Points used: [(0.0, 25.0), (10.0, 26.7), (20.0, 29.4), (35.0, 33.2), (50.0, 35.5), (65.0, 36.1), (80.0, 37.8), (90.0, 38.9), (100.0, 40.0)]
    Lagrange T(45) = 35.0063, Δ = 3.5872e-02
    Newton T(45)   = 35.0063, Δ = 3.5872e-02
    Lagrange Poly: 5.784339111767213e-13*x**8 - 2.22228388895053e-10*x**7 + 3.43577101910425e-8*x**6 - 2.74798272714936e-6*x**5 + 0.000122918436501768*x**4 - 0.003168102376019
  03*x**3 + 0.0458955942205928*x**2 - 0.0708036408036365*x + 25.0
    Newton Poly:   5.784339111767238e-13*x**8 - 2.22228388895053e-10*x**7 + 3.43577101910431e-8*x**6 - 2.74798272714936e-6*x**5 + 0.000122918436501768*x**4 - 0.003168102376019
  *x**3 + 0.0458955942205936*x**2 - 0.0708036408036382*x + 25.0
```

Figure 9: Degree 7, 8 Output

```
Predicted temperatures at x = 45 km:
 Degree  Lagrange_T(45)  Newton_T(45)
      2       34.922222     34.922222
      3       34.912346     34.912346
      4       34.974074     34.974074
      5       34.956117     34.956117
      6       34.943124     34.943124
      7       34.970378     34.970378
      8       35.006250     35.006250
```

Figure 10: Output

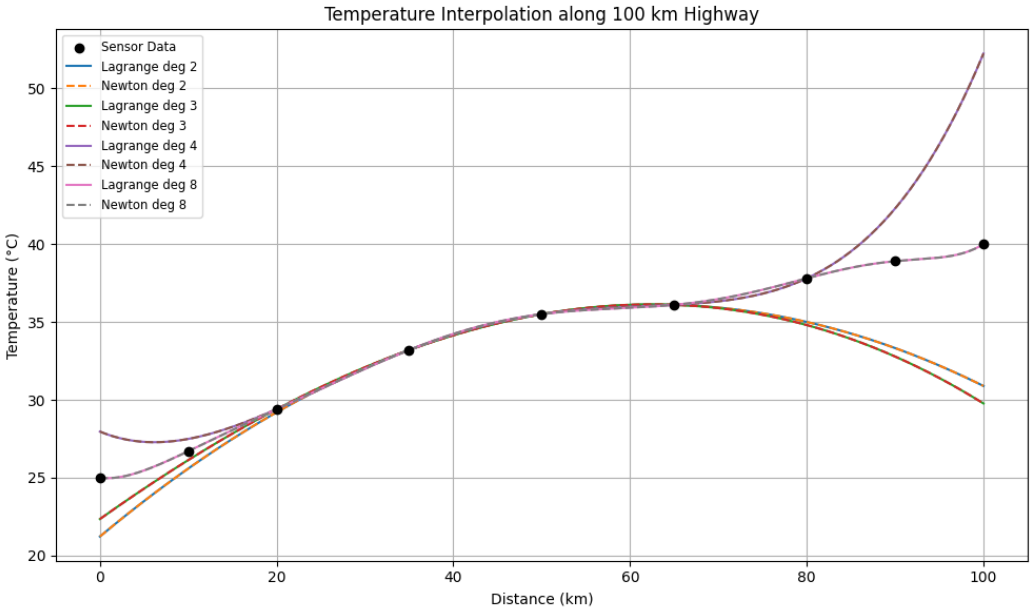

Figure 11: Interpolation curve Graph

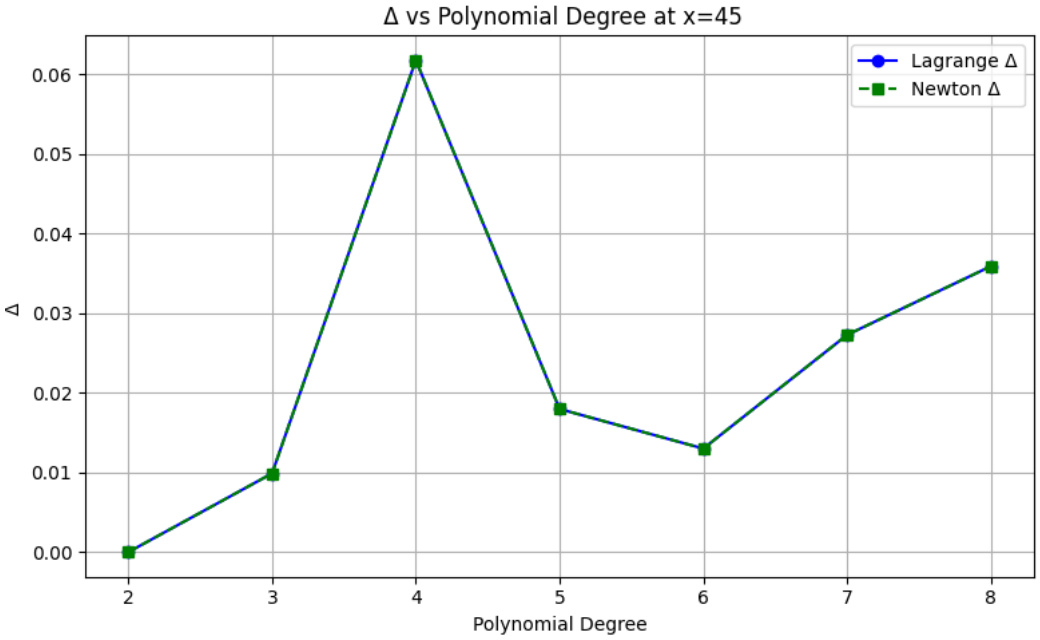

Figure 12: Convergence graph of increasing degree

6 Summary

In this experiment, we studied and implemented two widely used interpolation techniques: **Lagrange Interpolation** and **Newton's Divided Difference Interpolation**. Interpolation is a numerical method used to estimate the value of a function at points where no data is available, based on known discrete data points.

Using Lagrange Interpolation, we approximated values by constructing the Lagrange polynomial, which uses all given data points simultaneously. Newton's Divided Difference method, on the other hand, constructs the interpolation polynomial incrementally using divided differences, which can be more efficient when adding additional data points.

Through this exercise, we observed that both methods yield accurate estimates, but Newton's method generally requires less computation for incremental data and is suitable for constructing forward or backward interpolation polynomials. Overall, the experiment demonstrated the practical importance of interpolation in estimating unknown values and highlighted the trade-offs between computational complexity and ease of implementation for the two methods.